

Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory

Assignment No. 8 — Graph Algorithms Part-1

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

1. Problem Understanding

The assignment asks for the shortest circular route in an undirected weighted graph. The route must return to its starting town and must not use any road more than once. It need not visit all towns. The input contains up to 1000 roads and no more than 300 towns. The assignment guarantees that at least one circular route exists.

The assignment hint specifically suggests temporarily deleting an edge, finding the shortest path between its endpoints, and adding the deleted edge back to obtain a candidate cycle.

Approach 1: Brute-Force DFS / Cycle Enumeration

- Start DFS from each vertex and explore all possible paths. Whenever the DFS returns to its starting vertex, a cycle has been found.
- Maintain a visited-edge structure so that the same road is never used twice in one route. The minimum length among all discovered cycles is the answer.
- This approach directly follows the definition of the problem, but the number of possible paths/cycles can grow exponentially with the number of edges.
- It is useful mainly for understanding or validating small graphs; it is not suitable for the assignment's maximum input size.

Time Complexity: Exponential in the worst case (can be approximately $O(V!)$ -scale for dense/simple-cycle enumeration).

Space Complexity: $O(V + E)$ for graph storage, recursion/visited structures.

Approach 2: Edge-Deletion + Bellman-Ford

- For every edge (u, v, w) , temporarily ignore that edge and compute the shortest path from u to v .
- Use Bellman-Ford for the shortest-path calculation. If v is reachable, $w + \text{dist}(u, v)$ gives a candidate cycle containing the removed edge.
- Repeat for every edge and take the minimum candidate. This is conceptually the same edge-deletion idea as the assignment hint, but uses a different shortest-path algorithm.
- Because the assignment road lengths are positive, Bellman-Ford is valid but does substantially more work than necessary.

Time Complexity: $O(E \cdot V \cdot E) = O(E^2V)$, because Bellman-Ford is $O(VE)$ for each of E removed edges.

Space Complexity: $O(V + E)$.

Approach 3: Floyd-Warshall Minimum-Cycle Method

- Use dynamic programming to maintain shortest paths whose internal vertices belong to an increasingly larger prefix of the vertex set.
- Before allowing vertex k as an intermediate vertex, examine cycles that use k as the highest-indexed internal vertex: a path from i to j using previous vertices plus edges (i,k) and (k,j) forms a cycle.
- For every valid i, j, k combination, update the global minimum cycle length. Then perform the normal Floyd-Warshall relaxation using k .
- This approach is especially attractive when V is small or the graph is dense. Here $V \leq 300$, so $O(V^3)$ is feasible.

Time Complexity: $O(V^3)$.

Space Complexity: $O(V^2)$.

Approach 4: Edge-Deletion + Dijkstra (Recommended / Optimal for These Constraints)

- For every undirected edge (u, v, w) , temporarily exclude exactly that edge from the graph and run Dijkstra from u to v .
- All road lengths are positive, so Dijkstra correctly computes the shortest alternate path after the edge is removed.
- If an alternate path exists, adding w to that shortest-path distance gives the shortest cycle containing that edge. Taking the minimum over all edges gives the shortest cycle in the graph.
- An edge ID is used rather than only checking endpoints, so the exact road being removed is skipped even if the graph contains parallel roads.

Time Complexity: $O(E \cdot (E + V) \log V)$, commonly written as $O(E^2 \log V)$ for this sparse-graph setting.

Space Complexity: $O(V + E)$.

2. Comparison of All Approaches:

Approach	Main Idea	Time Complexity	Space Complexity	Suitability
1. Brute-Force DFS	Enumerate possible cycles directly	Exponential / $O(V!)$ -scale worst case	$O(V+E)$	Only very small graphs
2. Edge-Deletion + Bellman-Ford	Remove each edge; shortest path with Bellman-Ford	$O(E^2V)$	$O(V+E)$	Correct but unnecessarily slow
3. Floyd-Warshall	DP over all vertex pairs and detect minimum cycle	$O(V^3)$	$O(V^2)$	Very good when V is small/dense
4. Edge-Deletion + Dijkstra	Remove each edge; shortest alternate path with Dijkstra	$O(E(E+V) \log V)$	$O(V+E)$	Best fit for this assignment

3. Why Approach 3 is Selected

- **Efficient for Given Constraints:** With at most 300 towns, Floyd–Warshall's $O(V^3)$ complexity is practical for the assignment.
- **Finds Shortest Paths Globally:** It computes shortest paths between all pairs of vertices, making it suitable for identifying the minimum circular route.
- **Better Runtime:** Unlike repeatedly running Dijkstra for every edge, Floyd–Warshall avoids multiple priority-queue searches and provides predictable execution time.
- **Simple and Reliable:** The algorithm uses a distance matrix and systematic relaxation, making the implementation easier to understand, verify, and maintain for the given graph constraints.

Code :

```
#include<bits/stdc++.h>
using namespace std;

using ll=long long;
const ll INF=(1LL<<62);

struct Edge{
int u,v;
ll w;
};

ll solve(istream&in){
int M;
```

```

in>>M;

vector<Edge>edges(M);
int V=0;

for(int i=0;i<M;++i){
in>>edges[i].u>>edges[i].v>>edges[i].w;
V=max(V,max(edges[i].u,edges[i].v));
}

vector<vector<pair<int,int>>>adj(V+1);

for(int id=0;id<M;++id){
adj[edges[id].u].push_back({edges[id].v,id});
adj[edges[id].v].push_back({edges[id].u,id});
}

ll answer=INF;
vector<ll>dist(V+1);
vector<int>parentEdge(V+1);

for(int s=1;s<=V;++s){
fill(dist.begin(),dist.end(),INF);
fill(parentEdge.begin(),parentEdge.end(),-1);

priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>>pq;

dist[s]=0;
pq.push({0,s});

while(!pq.empty()){
auto[du,u]=pq.top();
pq.pop();

if(du!=dist[u])
continue;

for(auto[v,id]:adj[u]){
const Edge&e=edges[id];
ll nd=du+e.w;

if(nd<dist[v]){
dist[v]=nd;
parentEdge[v]=id;
}
}
}

```

```

pq.push({nd,v});
}
}
}

for(int id=0;id<M;++id){
const Edge&e=edges[id];

if(parentEdge[e.u]==id || parentEdge[e.v]==id)
continue;

if(dist[e.u]==INF || dist[e.v]==INF)
continue;

ll candidate=dist[e.u]+e.w+dist[e.v];
answer=min(answer,candidate);
}
}

return answer;
}

```

```

// CHP JUDGE MAIN
// int main(){
// ios::sync_with_stdio(false);
// cin.tie(nullptr);

// cout<<solve(cin)<<"\n";

// return 0;
// }

```

```

int main(){
ios::sync_with_stdio(false);
cin.tie(nullptr);

ifstream fin("testcases.txt");
ofstream fout("output.txt");

if(!fin){
cerr<<"Error: testcases.txt not found\n";
return 1;
}

```

```

if(!fout){
    cerr<<"Error: output.txt could not be created\n";
    return 1;
}

int T;
fin>>T;

for(int tc=1;tc<=T;++tc){
    fout<<solve(fin)<<'\\n';
}

fin.close();
fout.close();

cout<<"Processed "<<T<<" test cases. Results written to output.txt\\n";

return 0;
}

```

Output:

```

┌ *Linux@atharvk main in ~/Documents/WCE_Lab_Material
└─> g++ -std=c++17 -O2 padayatra.cpp -o padayatra
./padayatra
Processed 23 test cases. Results written to output.txt

```

output.txt

```
1 20297
2 47681
3 15205
4 25631
5 10092
6 24
7 30
8 36
9 42
10 48
11 23890
12 22245
13 59566
14 24965
15 24939
16 30090
17 7596
18 6046
19 16566
20 13716
21 12
22 77
23 116
24
```

expected_output.txt

```
1 Test Case 1: 20297
2 Test Case 2: 47681
3 Test Case 3: 15205
4 Test Case 4: 25631
5 Test Case 5: 10092
6 Test Case 6: 24
7 Test Case 7: 30
8 Test Case 8: 36
9 Test Case 9: 42
10 Test Case 10: 48
11 Test Case 11: 23890
12 Test Case 12: 22245
13 Test Case 13: 59566
14 Test Case 14: 24965
15 Test Case 15: 24939
16 Test Case 16: 30090
17 Test Case 17: 7596
18 Test Case 18: 6046
19 Test Case 19: 16566
20 Test Case 20: 13716
21 Test Case 21: 12
22 Test Case 22: 77
23 Test Case 23: 116
24
```

Local: padayatra_FINAL_CHP		23 / 23 passed	
✓ TC 1	Passed	5ms	 
✓ TC 2	Passed	3ms	 
✓ TC 3	Passed	42ms	 
✓ TC 4	Passed	40ms	 
✓ TC 5	Passed	104ms	 
✓ TC 6	Passed	30ms	 
✓ TC 7	Passed	28ms	 
✓ TC 8	Passed	18ms	 
✓ TC 9	Passed	18ms	 
✓ TC 10	Passed	18ms	 
✓ TC 11	Passed	52ms	 
✓ TC 12	Passed	51ms	 
✓ TC 13	Passed	28ms	 
✓ TC 14	Passed	30ms	 
✓ TC 15	Passed	26ms	 
✓ TC 16	Passed	27ms	 
✓ TC 17	Passed	54ms	 
✓ TC 18	Passed	51ms	 
✓ TC 19	Passed	95ms	 
✓ TC 20	Passed	76ms	 
✓ TC 21	Passed	2ms	 
✓ TC 22	Passed	2ms	 
✓ TC 23	Passed	2ms	 

